ASSIGNMENT ON RECURSION AND DIVIDE-AND-CONQUER PROBLEM SOLVING

Subject: Computer Science / Problem Solving and Programming
Topic: Recursion, Recursive Palindrome Checking, and Divide-and-Conquer
Submitted by: HARDISH M
Register Number: 192511098
Department: CSE

1. WHAT IS RECURSION? EXPLAIN ITS BASIC CONCEPT

1.1 Introduction

Recursion is an important programming technique used to solve problems by breaking them into smaller and simpler versions of the same problem. It is widely used in programming languages such as C, C++, Java, and Python.

In recursion, a function calls itself to solve a problem. Each recursive call works with a smaller or simpler input until a condition is satisfied that allows the function to stop calling itself.

Recursion is particularly useful when a problem can naturally be divided into smaller subproblems, such as calculating factorials, checking palindromes, traversing trees, and implementing sorting algorithms.

1.2 Definition of Recursion

Recursion is a programming technique in which a function calls itself, directly or indirectly, to solve a problem by reducing it into smaller instances of the same problem.

For recursion to work correctly, the function must have a stopping condition and a method of reducing the problem at each step.

1.3 Basic Concepts of Recursion

Recursion consists of two fundamental components.

A. Base Case

The base case is the condition that terminates the recursive process.

When the base case is satisfied, the function returns a result without making another recursive call.

Example:

```
IF n = 0 THEN
   RETURN 1
```

In a factorial calculation, the factorial of zero is defined as 1. Therefore, when the input becomes zero, the function stops making recursive calls.

Without a base case, a recursive function may continue calling itself until the program runs out of available stack memory.

B. Recursive Case

The recursive case is the part of the function in which it calls itself with a smaller or simpler input.

Example:

RETURN n * factorial(n - 1)

Here, the function calls itself using n - 1. The input becomes smaller with every call until it reaches the base case.

Thus, the base case stops recursion, while the recursive case moves the problem toward that stopping condition.

1.4 Working Principle of Recursion

Recursion generally works through the following steps:

1. The function receives an input.
2. It checks whether the base case has been reached.
3. If the base case is not satisfied, the function calls itself with a smaller input.
4. This process continues until the base case is reached.
5. The functions return their results in reverse order of their calls.
6. The final result is obtained after all required recursive calls have completed.

The computer manages these calls using a data structure called the **call stack**. Each function call creates a stack frame containing information needed to resume execution when that call returns.

1.5 Example: Factorial Using Recursion

The factorial of a positive integer is the product of all positive integers from 1 up to that number.

The factorial of a number $n$is represented as $n!$.

For example:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

The mathematical definition of factorial is:

$$n! = \begin{cases} 1, & \text{if } n = 0 \\ n \times (n-1)!, & \text{if } n > 0 \end{cases}$$

The first condition is the base case, and the second condition is the recursive case.

Algorithm

ALGORITHM Factorial(n)

1. BEGIN
2. IF n = 0 OR n = 1 THEN
3. RETURN 1
4. ELSE
5. RETURN n * Factorial(n - 1)
6. END IF
7. END

Pseudocode Explanation

- If n is 0 or 1, return 1.
- Otherwise, multiply n by the factorial of n - 1.
- Repeat the process until the base case is reached.
- Return the calculated result.

Step-by-Step Execution

Consider the input n = 5.

The recursive calls are:

Factorial(5)

= 5 × Factorial(4)

= 5 × 4 × Factorial(3)

= 5 × 4 × 3 × Factorial(2)

= 5 × 4 × 3 × 2 × Factorial(1)

= 5 × 4 × 3 × 2 × 1

= 120

The calls first move toward the base case. After Factorial(1) returns 1, the pending multiplications are performed.

Therefore:

Output: 120

Recursion Trace

Function Call	Operation	Result
Factorial(5)	5 × Factorial(4)	120
Factorial(4)	4 × Factorial(3)	24
Factorial(3)	3 × Factorial(2)	6
Factorial(2)	2 × Factorial(1)	2
Factorial(1)	Base case	1

The results are returned in reverse order, starting from the base case.

1.6 Advantages of Recursion

1. **Simplifies complex problems:** Large problems can be expressed as smaller versions of the same problem.
2. **Improves code readability:** Recursive solutions can be more concise than iterative solutions for certain problems.
3. **Useful for hierarchical structures:** Recursion is suitable for processing trees, file systems, and nested structures.
4. **Supports divide-and-conquer algorithms:** Many efficient algorithms use recursion to divide problems into smaller subproblems.
5. **Reduces complicated control flow:** Some algorithms become easier to understand when expressed recursively.

1.7 Limitations of Recursion

1. **Memory usage:** Each recursive call requires stack memory.
2. **Performance overhead:** Function calls introduce additional execution overhead.
3. **Risk of infinite recursion:** An incorrect or unreachable base case may cause the program to exhaust its stack memory.
4. **Difficulty in tracing:** Deep recursion may be difficult to debug and understand.
5. **Not always the most efficient approach:** Some recursive algorithms repeat calculations unnecessarily.

1.8 Applications of Recursion

Recursion is used in:

- Factorial calculation.
- Fibonacci sequence generation.
- Palindrome checking.
- Binary search.
- Merge sort and quick sort.
- Tree traversal.
- Tower of Hanoi.
- Backtracking problems such as maze solving.

1.9 Conclusion

Recursion is a powerful problem-solving technique in which a function calls itself to solve smaller instances of a problem. Its two essential components are the base case and the recursive case. A properly designed recursive function gradually reduces the problem until the base case is reached and then returns the result. Recursion is especially useful for problems that have naturally repetitive or hierarchical structures.

2. DESIGN AN ALGORITHM FOR PALINDROME CHECKING USING RECURSION

2.1 Introduction

A palindrome is a word, number, or sequence of characters that reads the same forward and backward.

Palindrome checking is a common application of recursion because a string can be checked by comparing its outermost characters and then applying the same procedure to the remaining inner characters.

For example, the word MADAM is a palindrome because reading it from left to right or right to left produces the same word.

2.2 Definition of a Palindrome

A palindrome is a sequence that remains unchanged when its characters are read in reverse order.

Examples

Input	**Reverse**	**Result**
MADAM	MADAM	Palindrome
LEVEL	LEVEL	Palindrome
RADAR	RADAR	Palindrome
HELLO	OLLEH	Not a palindrome
COMPUTER	RETUPMOC	Not a palindrome
121	121	Palindrome

For this algorithm, character comparisons are case-sensitive, and spaces and punctuation are treated as ordinary characters. Therefore, Madam is not considered a palindrome under the stated rules because M and m are different characters.

2.3 Principle of Recursive Palindrome Checking

The recursive palindrome algorithm compares the first and last characters of a string.

There are three possible situations:

Case 1: The Base Case Is Reached

If the left index is greater than or equal to the right index, all necessary character comparisons have succeeded.

The string is a palindrome, so the function returns TRUE.

Case 2: The Characters Do Not Match

If the leftmost character and the rightmost character are different, the string is not a palindrome.

The function returns FALSE immediately.

Case 3: The Characters Match

If the two characters are equal, the algorithm moves inward by increasing the left index and decreasing the right index.

It then calls itself recursively to check the remaining substring.

This process continues until a mismatch is found or the base case is reached.

2.4 Algorithm for Recursive Palindrome Checking

Step-by-Step Algorithm

Step 1: Start.

Step 2: Read the input string S.

Step 3: Calculate the length of the string.

Step 4: Initialize the left index to 0.

Step 5: Initialize the right index to length(S) - 1.

Step 6: Call the recursive function Palindrome(S, left, right).

Step 7: Inside the recursive function:

- If left >= right, return TRUE.
- If S[left] != S[right], return FALSE.
- Otherwise, recursively call Palindrome(S, left + 1, right - 1).

Step 8: If the function returns TRUE, display "The string is a palindrome."

Step 9: Otherwise, display "The string is not a palindrome."

Step 10: Stop.

2.5 Pseudocode

```
FUNCTION Palindrome(S, left, right)
    IF left >= right THEN
        RETURN TRUE
    END IF
    IF S[left] != S[right] THEN
        RETURN FALSE
    END IF
    RETURN Palindrome(S, left + 1, right - 1)
END FUNCTION
ALGORITHM Main
    READ S
    left = 0
    right = LENGTH(S) - 1
    result = Palindrome(S, left, right)
    IF result = TRUE THEN
        PRINT "The string is a palindrome"
    ELSE
        PRINT "The string is not a palindrome"
 END IF
END ALGORITHM
```

2.6 Explanation of the Algorithm

The recursive function receives three parameters:

- S: The input string to be checked.
- left: The index of the character being checked from the beginning.

- right: The index of the character being checked from the end.

Initially, left is set to zero because string indexing begins at zero in this pseudocode, and right is set to the final character's index.

The function first checks whether left >= right. If this condition is true, no further comparison is required, and the function returns TRUE.

Otherwise, the function compares S[left] with S[right].

If they are different, the function returns FALSE. If they are equal, the function calls itself with left + 1 and right - 1.

This reduces the size of the problem by excluding the two characters that have already been checked.

The process continues until the string is confirmed to be a palindrome or a mismatch is detected.

2.7 Example 1: Checking the String "MADAM"

Input:

S = "MADAM"

The length of the string is 5.

Therefore:

left = 0

right = 4

The characters and their indices are:

Index 0 1 2 3 4

Character M A D A M

Step-by-Step Execution

First Recursive Call

Palindrome("MADAM", 0, 4)

The function compares:

S[0] = M

S[4] = M

Both characters match.

The function calls itself with the updated indices:

Palindrome("MADAM", 1, 3)

Second Recursive Call

Palindrome("MADAM", 1, 3)

The function compares:

S[1] = A

S[3] = A

Both characters match.

The function calls itself again:

Palindrome("MADAM", 2, 2)

Third Recursive Call

Palindrome("MADAM", 2, 2)

Now:

left = 2

right = 2

Since left >= right, the base case is satisfied.

The function returns TRUE.

Execution Trace

Palindrome("MADAM", 0, 4)

|

M = M

|

Palindrome("MADAM", 1, 3)

|

A = A

|

Palindrome("MADAM", 2, 2)

|

Base Case

|

Return TRUE

Output:

The string is a palindrome.

Therefore, MADAM is a palindrome.

2.8 Example 2: Checking the String "HELLO"

Input:

S = "HELLO"

The initial indices are:

left = 0

right = 4

The function compares:

S[0] = H

S[4] = O

Since H and O are different, the function immediately returns FALSE.

No additional recursive calls are required.

Output:

The string is not a palindrome.

Therefore, HELLO is not a palindrome.

2.9 Example 3: Checking an Even-Length Palindrome

Consider the string:

S = "NOON"

The first and last characters are both N, so the algorithm moves inward.

Next, the characters at indices 1 and 2 are both O.

After this comparison, the left index becomes 2 and the right index becomes 1.

Since left >= right, the base case is satisfied.

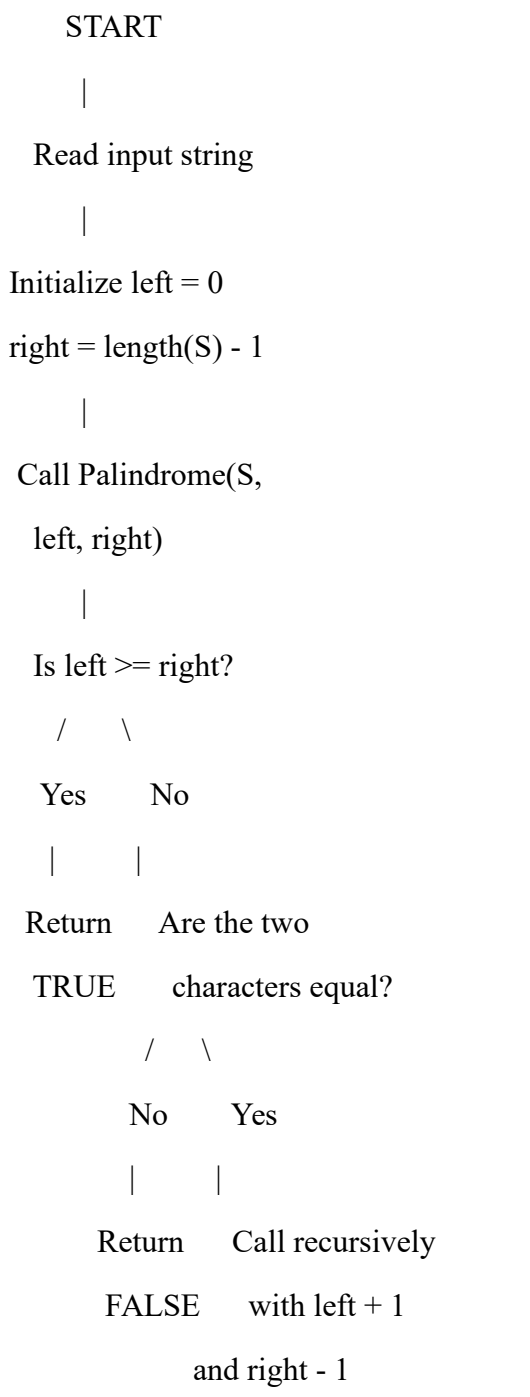
Output:

The string is a palindrome.

This demonstrates that the algorithm works for both odd-length and even-length strings.

2.10 Flow of the Algorithm

The logical flow of recursive palindrome checking is:



|

Repeat the checks

|

Display the result

| STOP

2.11 Time and Space Complexity

Let $n$represent the length of the input string.

Time Complexity

In the worst case, the algorithm compares approximately half the characters in the string.

The number of comparisons is proportional to $n/2$.

Ignoring constant factors:

$$T(n) = O(n)$$

Therefore, the worst-case time complexity is **O(n)**.

If the first and last characters do not match, the algorithm terminates immediately. Its best-case time complexity is $O(1)$.

Space Complexity

Each recursive call uses stack memory.

For a string of length n, the maximum recursion depth is approximately $n/2$.

Therefore, the auxiliary space complexity is:

$$S(n) = O(n)$$

The complexity is linear because the maximum number of simultaneous recursive calls grows proportionally to the input length.

2.12 Advantages of Recursive Palindrome Checking

1. **Simple implementation:** The algorithm follows a straightforward comparison process.
2. **No string reversal required:** It checks the characters directly from both ends.
3. **Early termination:** The algorithm stops as soon as a mismatch is detected.
4. **Works for odd and even lengths:** The base condition handles both cases.
5. **Demonstrates recursion clearly:** Each recursive call solves a smaller version of the original problem.

2.13 Limitations

1. Recursive calls consume stack memory.
2. Deep recursion may cause a stack overflow for very large strings in environments with limited stack space.
3. Character case, spaces, and punctuation must be handled explicitly if the application requires case-insensitive or normalized palindrome checking.

2.14 Conclusion

Recursive palindrome checking is an effective application of recursion. The algorithm compares the outermost characters and then recursively checks the remaining inner characters. If all corresponding characters match, the string is a palindrome. If any pair differs, the string is not a palindrome. The algorithm has a worst-case time complexity of $O(n)$and an auxiliary space complexity of $O(n)$.

3. DISCUSS RECURSION IN PROBLEM SOLVING (DIVIDE AND CONQUER)

3.1 Introduction

Divide and conquer is a widely used algorithm design technique for solving complex problems efficiently. It works by dividing a large problem into smaller subproblems, solving those subproblems, and combining their results to obtain the final solution.

Recursion is commonly used to implement divide-and-conquer algorithms because the same process can be applied repeatedly to each smaller subproblem.

Examples include merge sort, quick sort, and binary search.

3.2 Definition of Divide and Conquer

Divide and conquer is a problem-solving technique in which a problem is divided into smaller subproblems, each subproblem is solved recursively, and the solutions are combined to produce the final result.

The technique generally consists of three stages:

1. Divide.
2. Conquer.
3. Combine.

3.3 Three Main Steps of Divide and Conquer

A. Divide

The original problem is divided into smaller subproblems.

These subproblems are generally smaller instances of the original problem.

For example, an array containing eight elements can be divided into two smaller arrays, each containing four elements.

B. Conquer

The smaller subproblems are solved.

If a subproblem is still too large to solve directly, the algorithm applies recursion to divide it further.

When a subproblem becomes sufficiently simple, a base case is used to solve it directly.

For example, in merge sort, an array containing only one element is already sorted. Therefore, no further division is necessary.

C. Combine

After solving the subproblems, their results are combined to produce the solution to the original problem.

For example, merge sort combines two sorted subarrays into a single sorted array.

Not every divide-and-conquer algorithm requires a complicated combination step. In binary search, the algorithm repeatedly selects a smaller search interval, and no merging of separate solutions is needed.

3.4 Relationship Between Recursion and Divide and Conquer

Recursion and divide and conquer are closely related, but they are not identical.

Recursion is a programming technique in which a function calls itself.

Divide and conquer is an algorithm design strategy that breaks a problem into smaller subproblems.

Divide-and-conquer algorithms often use recursion to solve the subproblems. However, recursion can also be used for problems that do not follow divide and conquer, such as calculating a factorial.

The relationship can be summarized as follows:

Recursion	**Divide and Conquer**
A function calls itself.	A problem is divided into smaller subproblems.
Uses a base case to stop recursion.	Uses simple cases to solve small subproblems.
Focuses on the process of repeated function calls.	Focuses on the strategy for solving a problem.
Used in factorial and tree traversal.	Used in merge sort, quick sort, and binary search.

3.5 Example: Merge Sort Using Divide and Conquer

Merge sort is a sorting algorithm that arranges elements in ascending or descending order.

It uses the divide-and-conquer technique to divide an array into smaller subarrays, sort them recursively, and merge the sorted results.

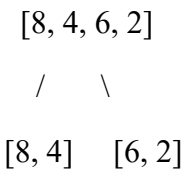
Consider the following array:

[8, 4, 6, 2]

The objective is to arrange the elements in ascending order.

Step 1: Divide

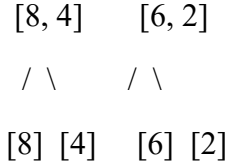
The original array is divided into two halves.



The array has now been divided into two smaller subarrays.

Step 2: Divide Further

Each subarray is divided again.



Each subarray now contains only one element.

An array with one element is already sorted. Therefore, the base case has been reached.

Step 3: Conquer and Combine

The individual elements are merged into sorted pairs.

[8] + [4] → [4, 8]

[6] + [2] → [2, 6]

The two sorted pairs are then merged into one sorted array.

[4, 8] + [2, 6] → [2, 4, 6, 8]

Final Output

[2, 4, 6, 8]

The array is now sorted in ascending order.

Explanation of the Example

The example demonstrates all three stages:

- **Divide:** Split the original array into smaller arrays.
- **Conquer:** Continue dividing until each subarray contains one element.
- **Combine:** Merge the smaller sorted arrays into larger sorted arrays until the entire array is sorted.

Recursion makes it possible to apply the same procedure to each subarray without writing a separate algorithm for every possible array size.

3.6 General Pseudocode for Divide and Conquer

```
ALGORITHM DivideAndConquer(problem)
  IF problem is small enough THEN
    RETURN SolveDirectly(problem)
  END IF
  Divide problem into smaller subproblems
  FOR EACH subproblem
    Solve the subproblem recursively
  END FOR
  Combine the results of the subproblems
  RETURN final result
END ALGORITHM
```

This is a general representation of divide and conquer. The exact division and combination steps depend on the problem being solved.

3.7 Other Applications of Divide and Conquer

A. Quick Sort

Quick sort selects a pivot element and partitions the array into smaller groups. It then recursively sorts the groups.

Its average time complexity is $O(n \log n)$, while its worst-case time complexity is $O(n^2)$.

B. Binary Search

Binary search finds an element in a sorted array by repeatedly examining the middle element and eliminating half of the remaining search space.

Its time complexity is $O(\log n)$.

C. Merge Sort

Merge sort repeatedly divides an array into halves and merges the sorted subarrays.

Its time complexity is $O(n \log n)$in the best, average, and worst cases.

3.8 Advantages of Divide and Conquer

1. **Efficient problem solving:** It can reduce the amount of work required for many problems.
2. **Simplifies complex problems:** Large problems become easier to manage when divided into smaller parts.
3. **Supports recursive design:** The same algorithm can be applied to each subproblem.
4. **Suitable for parallel processing:** Independent subproblems can sometimes be solved simultaneously.
5. **Useful for large datasets:** Algorithms such as merge sort and binary search are effective for processing large amounts of data.

3.9 Limitations of Divide and Conquer

1. Recursive function calls may require additional stack memory.
2. Combining subproblem results can introduce extra computation.
3. The approach may be inefficient when subproblems repeatedly overlap and the same calculations are performed many times.
4. Designing the correct division and combination steps can be challenging for some problems.

3.10 Conclusion

Divide and conquer is an important algorithm design strategy that solves complex problems by dividing them into smaller subproblems, solving those subproblems, and combining their results. Recursion provides a natural way to implement this strategy. Merge sort, quick sort, and binary search are common examples. When appropriately designed, divide-and-conquer algorithms can improve efficiency and make complex problems easier to solve.

OVERALL CONCLUSION

Recursion is a fundamental programming technique in which a function calls itself to solve smaller instances of a problem. The base case terminates the recursive process, while the recursive case reduces the problem and moves it toward termination.

Recursive palindrome checking demonstrates how recursion can be used to compare characters from opposite ends of a string until a mismatch is found or all required comparisons are completed.

Divide and conquer extends the problem-solving approach by dividing a large problem into smaller subproblems, solving them, and combining their results. Algorithms such as merge sort and quick sort demonstrate how recursion can support efficient problem solving.

Therefore, recursion is an important concept in computer science because it provides a structured approach to solving problems involving repeated operations, hierarchical structures, and the decomposition of complex tasks into simpler subproblems.